



Universidade do Minho
Escola de Engenharia

Universidade do Minho
Escola de Engenharia

Processamento de Linguagens

Trabalho prático

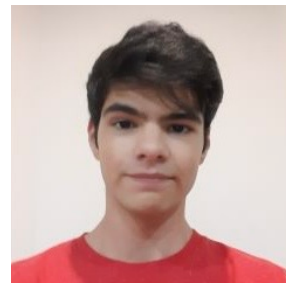
Grupo 49



Diogo Linhares Campos
(A106920)



Nelson Daniel Araújo Sousa
(A109068)



Duarte Alexandre Oliveira Faria
(A95609)

1. Introdução

Este projeto consistiu no desenvolvimento de um compilador simples para uma linguagem inspirada em Fortran. O sistema suporta um programa principal, funções, sub-rotinas, expressões, estruturas de controlo, leitura/escrita e geração de código para uma máquina virtual.

A implementação foi organizada em quatro fases principais: análise léxica, análise sintática, análise semântica e geração de código. Esta divisão permitiu manter cada componente focado numa responsabilidade específica, facilitando a implementação, a depuração e a manutenção do projeto.

2. Estrutura geral

O compilador encontra-se dividido em cinco ficheiros principais: `lexer.py`, `parser.py`, `semantic.py`, `codegen.py` e `main.py`. Cada um destes módulos corresponde a uma etapa do processo de compilação.

- `lexer.py` define os tokens da linguagem e reconhece palavras reservadas, identificadores, constantes e operadores.
- `parser.py` aplica a gramática e constrói a AST.
- `semantic.py` executa a análise semântica, com verificação de tipos, escopos e inicialização.
- `codegen.py` traduz a AST validada em instruções para a máquina virtual.
- `main.py` faz a ligação entre todas as fases e gera o ficheiro de saída `.vm`.

O fluxo de compilação corresponde a um pipeline em que a saída de cada fase é a entrada da fase seguinte:

```
+-----+ +-----+ +-----+ +-----+ +-----+
| Fonte | -> | Lexer | -> | Parser | -> | Semântica | -> | Codegen | -> .vm
| .f90  |   | tokens |   | AST   |   | AST valid. |   | instruções |
+-----+ +-----+ +-----+ +-----+ +-----+
```

Ao nível dos ficheiros, o projeto está organizado da seguinte forma:

```
PL-2526/
├─ src/
│   ├── lexer.py      # análise léxica
│   ├── parser.py     # análise sintática (constrói a AST)
│   ├── semantic.py   # análise semântica
│   ├── codegen.py    # geração de código
│   └── main.py       # ponto de entrada
└─ testes/           # programas de teste e respetivas saídas .vm
```

3. Análise léxica

O módulo léxico foi implementado com PLY e tem como objetivo reconhecer os elementos básicos da linguagem. Entre as palavras reservadas suportadas encontram-se `PROGRAM`, `END`, `INTEGER`, `REAL`, `LOGICAL`, `CHARACTER`, `IF`, `DO`, `READ`, `PRINT`, `FUNCTION`, `SUBROUTINE`, `CALL` e `RETURN`.

Também foram definidos operadores relacionais e lógicos no estilo Fortran, como `.EQ.`, `.NE.`, `.LT.`, `.LE.`, `.GT.`, `.GE.`, `.AND.`, `.OR.` e `.NOT.`. Além disso, o lexer reconhece identificadores, números inteiros, números reais, strings e símbolos como `+`, `-`, `*`, `/`, `=`, `(`, `)` e `,`.

O tratamento de comentários e de espaços em branco foi simplificado para não interferir na análise. O lexer também atualiza corretamente o número de linha, o que permite produzir mensagens de erro mais úteis em caso de falha.

A tabela seguinte resume as principais categorias de tokens reconhecidas:

Categoria	Tokens
Palavras reservadas	PROGRAM, END, INTEGER, REAL, LOGICAL, CHARACTER, IF, THEN, ELSE, ENDIF, DO, CONTINUE, GOTO, READ, PRINT, FUNCTION, SUBROUTINE, CALL, RETURN
Operadores relacionais	.EQ., .NE., .LT., .LE., .GT., .GE.
Operadores lógicos	.AND., .OR., .NOT.
Operadores aritméticos	+, -, *, /
Delimitadores	(,), ,
Atribuição	=
Literais	inteiros, reais, strings, .TRUE., .FALSE.
Identificadores	nomes de variáveis e de subprogramas

4. Gramática utilizada

A gramática implementada no parser cobre programas compostos por uma ou mais unidades, com suporte para programa principal, funções e sub-rotinas. A estrutura sintática aceita blocos com declarações, instruções e, nos casos aplicáveis, listas de parâmetros.

Programas:

```
program_file : program_file program_unit
            | program_unit
```

```
program_unit : main_program
            | function_def
            | subroutine_def
```

```
main_program : PROGRAM ID
             declarations
             statements
             END
```

```
function_def : type FUNCTION ID LPAREN param_list_opt RPAREN
             declarations
             statements
             END
```

```
subroutine_def : SUBROUTINE ID LPAREN param_list_opt RPAREN
               declarations
               statements
               END
            | SUBROUTINE ID
               declarations
               statements
               END
```

```
param_list_opt : param_list
```

```

        |  $\epsilon$ 

param_list    : param_list COMMA ID
               | ID

```

Declarações de variáveis:

```

declarations  : declarations declaration
               |  $\epsilon$ 

declaration   : type id_decl_list

type          : INTEGER
               | REAL
               | LOGICAL
               | CHARACTER
               | CHARACTER LPAREN INT_CONST RPAREN
               | CHARACTER TIMES INT_CONST

id_decl_list  : id_decl_list COMMA id_decl
               | id_decl

id_decl       : ID LPAREN INT_CONST RPAREN
               | ID LPAREN ID RPAREN
               | ID

```

Statements:

```

statements    : statements labeled_stmt
               |  $\epsilon$ 

labeled_stmt  : INT_CONST statement
               | statement

statement     : assign_stmt
               | if_stmt
               | do_stmt
               | goto_stmt
               | continue_stmt
               | print_stmt
               | read_stmt
               | call_stmt
               | return_stmt

assign_stmt   : ID EQUALS expr
               | ID LPAREN expr RPAREN EQUALS expr

if_stmt       : IF LPAREN expr RPAREN THEN
               statements
               ENDIF
               | IF LPAREN expr RPAREN THEN

```

```

        statements
    ELSE
        statements
    ENDIF
| IF LPAREN expr RPAREN statement

do_stmt      : DO INT_CONST ID EQUALS expr COMMA expr
              do_body do_end
| DO INT_CONST ID EQUALS expr COMMA expr COMMA expr
              do_body do_end
do_body      : do_body statement
| ε

do_end       : INT_CONST CONTINUE

goto_stmt    : GOTO INT_CONST

continue_stmt : CONTINUE

print_stmt   : PRINT TIMES COMMA expr_list
read_stmt    : READ  TIMES COMMA read_list

read_list    : read_list COMMA read_item
| read_item

read_item    : ID
| ID LPAREN expr RPAREN

call_stmt     : CALL ID
| CALL ID LPAREN expr_list RPAREN

return_stmt  : RETURN

expr          : expr OR expr
| expr AND expr
| NOT expr
| expr EQ expr | expr NE expr
| expr LT expr | expr LE expr
| expr GT expr | expr GE expr
| expr PLUS  expr
| expr MINUS expr
| expr TIMES expr
| expr DIVIDE expr
| MINUS expr
| LPAREN expr RPAREN
| ID LPAREN expr_list RPAREN
| ID
| INT_CONST
| REAL_CONST
| STRING
| TRUE
| FALSE

```

```
expr_list      : expr_list COMMA expr
                | expr
```

A gramática foi estruturada para suportar construções típicas de uma linguagem imperativa com subprogramas. A distinção entre chamadas a funções e chamadas a sub-rotinas é feita a nível sintático e posteriormente confirmada pela análise semântica.

5. Opções de implementação

Uma das principais decisões foi recorrer a uma AST intermédia, em vez de gerar código diretamente no parser. Esta abordagem tornou o compilador mais modular e facilitou a integração da análise semântica antes da geração final.

Outra decisão importante foi o uso de uma tabela de símbolos com apoio a scopes. O compilador distingue corretamente variáveis locais, parâmetros e entidades globais, o que é essencial para lidar com funções e sub-rotinas.

Também foi adotado um mecanismo de anotação da AST com índices de memória, permitindo que a geração de código recupere diretamente a informação necessária para produzir instruções como `STOREG`, `PUSHG`, `LOADN` e `STOREN`.

6. Análise semântica

A análise semântica verifica se o programa respeita as regras da linguagem antes de ser gerado código. Esta fase usa uma tabela de símbolos organizada em pilha, o que permite gerir corretamente escopos aninhados e separar o contexto global dos contextos locais.

Foram implementadas verificações para:

- declarações repetidas no mesmo scope;
- uso de identificadores não declarados;
- uso de variáveis não inicializadas;
- compatibilidade de tipos em atribuições e expressões;
- validação do tipo dos índices de arrays;
- validação do número e tipo de argumentos em chamadas;
- coerência de labels: instruções `GOTO` só podem saltar para labels que estejam efetivamente definidos no corpo do mesmo subprograma (incluindo o label terminal de um ciclo `DO N`). A correspondência entre o label do `DO N` e o `N CONTINUE` que o termina é, por sua vez, garantida ao nível da gramática.

A análise faz primeiro a recolha das assinaturas de funções e sub-rotinas e só depois analisa os respetivos corpos. Isto permite resolver chamadas recursivas e dependências entre unidades do programa.

Um aspeto relevante do projeto foi o tratamento das funções, em que o nome da função é usado como variável local de retorno. Esta solução é coerente com a semântica de linguagens inspiradas em Fortran e facilita a geração de código do valor devolvido.

7. Geração de código

A geração de código percorre a AST validada e traduz cada construção para instruções da máquina virtual. O gerador emite operações para controlo de fluxo, manipulação de pilha, leitura/escrita, chamadas a funções e acesso a arrays.

As instruções produzidas incluem, entre outras, `START`, `STOP`, `PUSHI`, `PUSHF`, `PUSHS`, `ADD`, `SUB`, `MUL`, `DIV`, `WRITEI`, `WRITES`, `READ`, `JUMP`, `JZ`, `CALL` e `RETURN`. A geração é feita de forma sequencial, com etiquetas automáticas para ciclos e condicionais.

No caso dos arrays, o endereço é calculado a partir da base e do índice, com o ajustamento necessário para a convenção da máquina virtual. Já nas chamadas a funções e sub-rotinas, os argumentos são preparados antes da instrução `CALL`, assegurando a passagem correta de parâmetros.

8. Dificuldades encontradas

Uma das maiores dificuldades foi garantir coerência entre as várias fases do compilador. Como a AST é partilhada entre o parser, a análise semântica e a geração de código, qualquer diferença na estrutura dos nós podia causar erros difíceis de depurar.

Outra dificuldade esteve relacionada com a gestão de escopos e com a distinção entre variáveis, parâmetros, funções e sub-rotinas. Este problema tornou-se especialmente importante em chamadas a subprogramas e em expressões com acesso a arrays.

Também foi necessário tratar com cuidado a semântica das funções, já que o nome da função funciona como local de retorno. Essa decisão obrigou a uma integração cuidadosa entre a tabela de símbolos, a análise semântica e o código gerado.

Por fim, a construção de mensagens de erro claras foi importante para facilitar a correção de programas inválidos. O compilador identifica erros léxicos, sintáticos e semânticos com indicação de linha, o que ajuda bastante durante a fase de testes.

9. Como executar

O ponto de entrada do projeto é o ficheiro `main.py`. A execução faz-se a partir da linha de comando, passando como argumento o ficheiro fonte a compilar.

Exemplo:

```
python src/main.py exemplo.f90
```

Se a compilação terminar com sucesso, é gerado automaticamente um ficheiro `.vm` com o mesmo nome base do ficheiro de entrada. Em caso de erro, o compilador apresenta uma mensagem correspondente à fase onde ocorreu a falha.

10. Exemplo de compilação

Para ilustrar o resultado completo do processo de compilação, considera-se o seguinte programa em Fortran, presente em `testes/hello.f90`:

```
PROGRAM HELLO
  PRINT *, 'Ola, Mundo!'
END
```

Após a execução do compilador, é produzido o seguinte código para a máquina virtual (`testes/hello.vm`):

```
START
PUSHS "Ola, Mundo!"
WRITES
PUSHS "\n"
WRITES
STOP
```

Cada instrução corresponde a um passo da execução: `START` marca o início do programa; `PUSHS` empilha uma string constante; `WRITES` imprime no ecrã o valor que está no topo da pilha; e `STOP` termina a execução. O segundo `PUSHS "\n"` seguido de `WRITES` é gerado automaticamente pelo compilador para produzir a quebra de linha implícita do `PRINT *` em Fortran.

Este pequeno exemplo permite observar, de forma compacta, a correspondência entre as construções da linguagem de alto nível e as primitivas de baixo nível da máquina virtual, tornando visível o trabalho de tradução efetuado pelo gerador de código.

Para uma demonstração com construções mais ricas (ciclo `DO`, leitura de inteiros e atribuição acumulativa), considera-se o cálculo de um fatorial, presente em `testes/fatorial.f90`:

```
PROGRAM FATORIAL
  INTEGER N, I, FAT

  PRINT *, 'Introduza um numero inteiro positivo:'
  READ *, N

  FAT = 1
  DO 10 I = 1, N
    FAT = FAT * I
10  CONTINUE

  PRINT *, 'Fatorial de ', N, ': ', FAT
END
```

O código gerado pelo compilador é:

```
START
PUSHI 0
PUSHI 0
PUSHI 0
PUSHS "Introduza um numero inteiro positivo:"
WRITES
PUSHS "\n"
WRITES
READ
ATOI
STOREG 0
PUSHI 1
STOREG 2
PUSHI 1
STOREG 1
LOOP1:
PUSHG 1
PUSHG 0
INFEQ
JZ ENDL00P2
PUSHG 2
PUSHG 1
MUL
STOREG 2
PUSHG 1
PUSHI 1
ADD
STOREG 1
JUMP LOOP1
ENDL00P2:
PUSHS "Fatorial de "
```

```

WRITES
PUSHG 0
WRITEI
PUSHS ": "
WRITES
PUSHG 2
WRITEI
PUSHS "\n"
WRITES
STOP

```

Os três `PUSHI 0` iniciais reservam espaço na pilha global para as variáveis declaradas (`N` , `I` e `FAT`). O ciclo `DO` é traduzido para um bloco com duas etiquetas, `LOOP1` e `ENDLOOP2` : o teste de paragem usa `INFEQ` (menor ou igual) seguido de `JZ` , e cada iteração termina com um `JUMP` de regresso ao início. O acumulador `FAT` é multiplicado pelo contador `I` em cada iteração com `MUL` , e o valor final é impresso com `WRITEI` .

11. Testes

Para garantir a qualidade do compilador, foi criado um conjunto de testes automatizado na diretoria `testes/` , dividido em dois grupos:

- **positivos/** — programas que devem compilar com sucesso e gerar o respetivo ficheiro `.vm` . Cobrem expressões aritméticas (`arith`), atribuições (`assign`), arrays (`array`), ciclos `DO` (`doloop`), condicionais `IF/ELSE` (`ifelse`), operadores lógicos (`logop`), funções com diferentes assinaturas (`func` , `func_inexpr` , `func_multi`) e sub-rotinas (`subrot` , `sub_multi` , `sub_noargs`).
- **negativos/** — programas que devem falhar com uma mensagem de erro específica. Cada teste tem um ficheiro `.expected` com a mensagem que se espera obter do compilador.

O ficheiro `run_tests.py` corre todos os testes automaticamente: para os positivos confirma que o `.vm` é gerado; para os negativos confirma que a saída do compilador contém a mensagem do `.expected` .

Algumas das mensagens de erro testadas são:

Teste	Mensagem esperada
undeclared	Undeclared identifier B
dupdecl	Duplicate declaration A
uninit	Uninitialized variable A
typemis	Type error
goto_undef	GOTO to undefined label 99
dolabel	DO label mismatch
wrongargs	expected 2 arguments
sub_wrongargs	Subroutine MOSTRA expected 1 arguments
sub_badtype	argument 1 expected int
call_on_func	QUAD is not a subroutine

Esta abordagem permitiu, durante o desenvolvimento, detetar rapidamente regressões sempre que se introduzia uma alteração numa das fases do compilador, e serviu também como documentação executável do conjunto de construções suportadas pela linguagem.

12. Conclusão

O projeto permitiu desenvolver um compilador funcional, dividido em fases bem definidas e com uma arquitetura limpa e extensível. A solução cobre as construções principais da linguagem e produz código intermédio para execução numa máquina virtual.

Do ponto de vista técnico, as decisões mais importantes foram a utilização de AST, a gestão de scopes com tabela de símbolos e a separação clara entre análise semântica e geração de código. Estas escolhas deram robustez ao projeto e facilitaram a sua evolução.

Como trabalho futuro, identificamos a implementação de otimizações simples sobre a AST ou sobre o código gerado, como a propagação de constantes e a eliminação de subexpressões redundantes, que permitiriam reduzir o número de instruções emitidas para a máquina virtual sem alterar a semântica do programa.